🐍 Python Basics — Structured for Beginners

1️⃣ Lists in Python

A **list** is an ordered, mutable collection of items.

colors = ["red", "green", "blue", "green"]

Common List Methods

Method	What it does
`append(x)`	Add item at the end
`insert(i, x)`	Insert at specific index
`pop(i)`	Remove and return item at index (default: last)
`remove(x)`	Remove first occurrence of value
`count(x)`	Count frequency
`sort()`	Sort list
`reverse()`	Reverse list
`extend(iterable)`	Add multiple elements

Examples

Append vs Extend
colors = ["red", "blue", "black"]

new_colors = ["cyan", "yellow"]

colors.append(new_colors)
# ['red', 'blue', 'black', ['cyan', 'yellow']]

colors.extend(new_colors)
# ['red', 'blue', 'black', 'cyan', 'yellow']

Key difference:

- `append()` → adds as single element
- `extend()` → merges elements

Remove Elements

colors = ["red", "green", "blue"]

colors.pop(1)   # removes 'green'
colors.remove("red") # removes by value

Count Frequency

colors = ["red", "green", "green"]
print(colors.count("green")) # 2

If value not present:

print(colors.count("ui")) # 0

Mutable Nature of Lists

Lists can be modified.

fav = ["green", "black", "yellow"]
fav[1] = "orange"
print(fav)
# ['green', 'orange', 'yellow']

Copying Lists (Important)

a = ["red", "white"]
b = a.copy()

b[0] = "yellow"

print(a) # unchanged
print(b) # modified

`==` vs `is`

print(a == b) # compares values
print(a is b) # compares memory location

- `==` → value equality
- `is` → identity (same object)

Sorting

nums = [10, 5, 1]
nums.sort()
nums.sort(reverse=True)

Iterating Lists

Using index

foods = ["tea", "cake", "rice"]

for i in range(len(foods)):
    print(foods[i])

Direct iteration (better)

for f in foods:
    print(f)

2️⃣ Basic Data Types

```
address = """line1
line2"""
number = 732
is_active = True
avg = 4.6
```

Check Type

```
print(type(address))   # str
print(type(number))    # int
print(type(is_active)) # bool
print(type(avg))       # float
```

3️⃣ Strings in Python

- Strings are **immutable** (cannot change characters directly).

```
text = "hello"
# text[0] = "p" ❌ ERROR
```

4️⃣ Common String Methods

```
word = "   luminarch Technolab Technohub   "
```

Case Conversion

```
print(word.upper())
print(word.casefold())   # stronger lowercase
```

Searching

```
print(word.index("lu"))   # error if not found
print(word.find("lu"))    # -1 if not found
```

print(word.rfind("ch"))   # search from right
print(word.count("Tech"))

Difference:

- `index()` → raises error if not found
- `find()` → returns `-1`

Character Checks

print(word.isalpha()) # only letters?
print(word.isdigit()) # only numbers?
print(word.isalnum()) # letters + numbers?

⚠️ Leading spaces will make these return `False`.

Start / End Checks

print(word.startswith("lu"))
print(word.endswith("hub"))

Remove Spaces

print(word.strip())   # both sides
print(word.lstrip()) # left only
print(word.rstrip()) # right only

Capitalize Example

text = "hello world"
new_text = text.capitalize()

print(text)   # original unchanged
print(new_text) # Hello world

Removing Tabs / Newlines

word_name = "\tluminar technolab\n"

clean = word_name.strip()
print(clean)

5️⃣ Useful Built-in Tools

`dir()` — See Available Methods

print(dir(list))

Shows all methods supported by lists.

✅ Key Beginner Takeaways

- Lists are **mutable**, strings are **immutable**
- Use:
  - `append()` → single item
  - `extend()` → multiple items
- Prefer:
  - `for item in list` over index loops
- Remember:
  - `==` compares values
  - `is` compares identity
- `find()` is safer than `index()` for beginners
